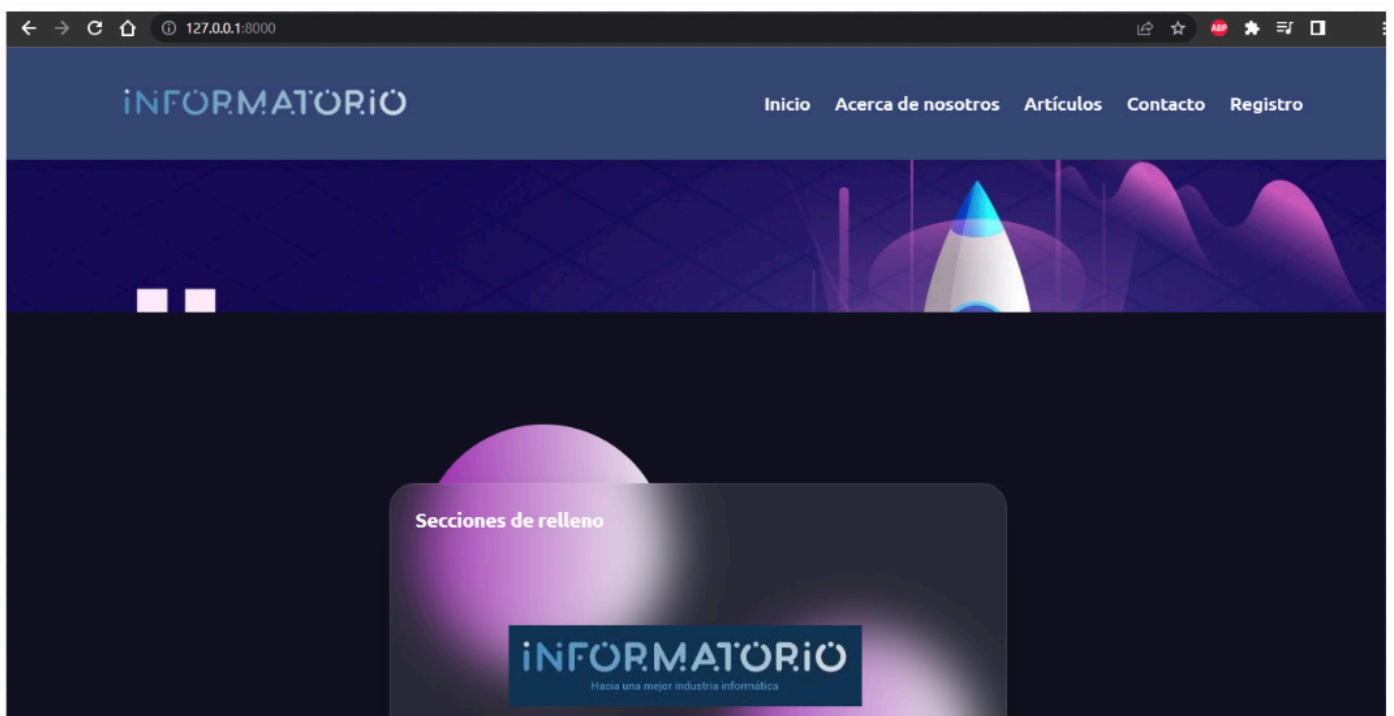
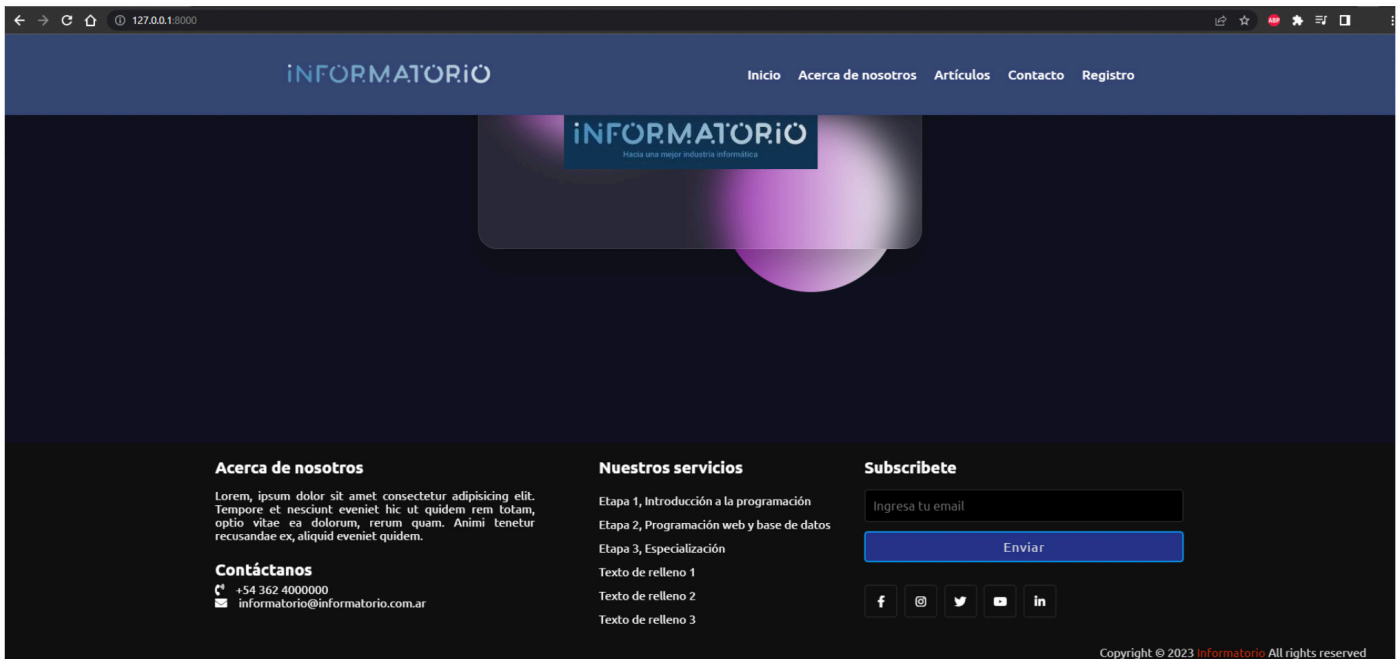


> Muestra frontend básica

Animándote a que
puedas modificar el
frontend de tus htmls





Aquí cerraremos esta parte. En la próxima te mostraremos lo que sería la vista de los "posts" con el html modificado, además te enseñaremos a incluir imagenes, ya que esto lleva más sintaxis y declaraciones (`{% load static %}` , `{{ posts.imagen.url }}`) entre otras cosas más. Además comenzaremos a crear otras aplicaciones.

Para que puedas seguir cada paso que vamos dando, es muy importante que respetes las sintaxis y no dejes pasar nada. Revisa varias veces si te da algún error al momento de ejecutar, porque de seguro solo son errores de sintaxis.

Este apunte se va realizando a medida que se va probando, por lo que si te surge algún error, es porque pusiste algo demás o de menos. Verifica bien cada cosa.

Hay cosas que pueden variar, pero otras que no, por lo que te recomendamos que hasta tener más práctica, sigas las mismas sintaxis que este apunte, así puedes seguir tal cual lo vas viendo aquí.

Además en las clases podrás incorporar otras cosas para usarlas en tu proyecto.

> Posts.html

Mostrando el html "post"

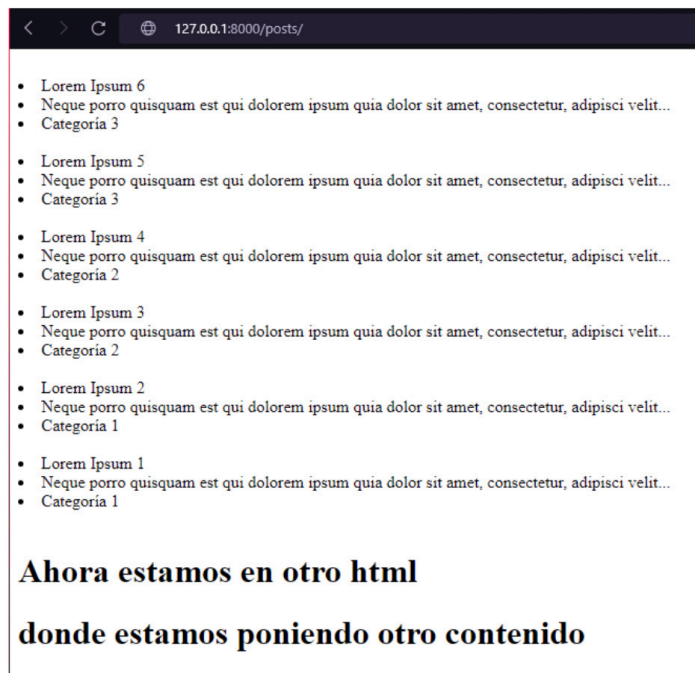


Pasos

Como te mencionamos en la parte anterior, vamos a mostrar "posts.html" desde el navegador. Vamos a usar el código que tenemos cargado y eso será suficiente para mostrar lo que hay en la base de datos.

Si lo pudiste ver, nosotros ya tenemos modificados nuestros html base con css y js para que tenga una mejor visual, sin embargo, esto no afectaría a lo que realmente tienes que hacer como trabajo backend. También te mencionamos que para hacer estas modificaciones e incluir css y js, utilizamos Bootstrap, tanto desde la extensión que instalamos en páginas anteriores, como desde la página de Bootstrap (<https://getbootstrap.com/>) la cual indica la instalación en el html base, haciendo declaraciones al principio del html. Puedes leer los pasos y documentación de esta página para lograr acabados similares o mejores a la página de prueba que te estamos mostrando.

Aclarado esto, vamos a "post.html" y antes de heredar de "base.html" como ya lo hicimos con "index", te vamos a mostrar como se vería la página sin hacer la herencia (ya que tenemos la vista creada y el url) accediendo a ella mediante <http://127.0.0.1:8000/posts/>



```
posts.html X
blog > templates > posts.html > ...
1  {% extends 'base.html' %}
2
3  {% block contenido %}
4
5
6      {% for i in posts %}
7
8          <br>
9          <li>{{ i.titulo }}</li>
10         <li>{{ i.subtitulo }}</li>
11         <li>{{ i.categoria }}</li>
12
13         {% empty %}
14         <h1>No hay registros</h1>
15
16     {% endfor %}
17
18     <br>
19     <h1>Ahora
20     estamos en
21     otro html</h1>
22     <h1>donde estamos
23     poniendo otro
24     contenido</h1>
25
26 {% endblock %}
```

> Vista de clases

Modificando nuestra vista basada en funciones por una de clases



Te dejamos la captura de "posts.html" (realizado en la página 45 y 46) para que puedas ver que con el código Python colocado dentro de la plantilla html podemos visualizar (accediendo mediante la url creada) los registros "posts" de nuestra base de datos (los que creamos con anterioridad desde el admin de Django).

Se puede observar también, que no tiene ningún formateo ni nada ya que no estamos heredando de base.html ni aplicando css ni js. y, por el momento, lo vamos a dejar así y al terminar de explicarte esta parte, aplicaremos la parte visual para que veas como se vería. Ahora, a modo de ejemplo y para que entiendas sin tanto código html de por medio, lo dejamos así.

Lo siguiente a esto, sería poder acceder a cada post de forma individual, para poder tener el contenido completo del texto, además, más adelante cuando creamos nuestros comentarios, será desde esta entrada individual al "post" que podremos generar a estos comentarios.

Para poder acceder a un post de manera individual, debemos hacer referencia al "id" del "post", y esto lo tenemos que hacer desde la url, previa creación de la view para esto, por lo que vamos a ir a nuestra view de la aplicación para extraer el id del post y poder ingresar a él.

Sin embargo, vamos a enlazar un tema más que son las **vistas basadas en Clases**.

Ya que tenemos nuestra vista para "posts" creada en base a funciones, ahora vamos a hacer la misma, para en base a Clases y lo hacemos de la siguiente manera:

```
apps > posts > views.py > ...
1  from django.shortcuts import render
2  from .models import Post
3  from django.views.generic import ListView
4
5  #Vista basada en funciones
6  def posts(request):
7      posts = Post.objects.all()
8      return render(request, 'posts.html', {'posts': posts})
9
10 #Vista basada en clases
11 class PostListView(ListView):
12     model = Post
13     template_name = "posts.html"
14     context_object_name = "posts"
15
```

Estas vistas hacen lo mismo (en este caso), muestran el contenido de la misma forma, sin embargo, al usar una vista basada en clases podremos realizar ciertas modificaciones sobrescribiendo o llamando algunos atributos propios de éstas vistas, sin tener que escribir tanto código como en la vista basada en funciones.

¿Cuando usaremos vistas basadas en funciones y cuando usaremos vistas basadas en clases?

Esto será según la complejidad de las vistas del proyecto y la capacidad que tengamos de saber usar las vistas basadas en clases.

> URL

Declarando la url – urls.py



Te recomendamos leer en la documentación de Django acerca de estas vistas (<https://docs.djangoproject.com/en/4.2/topics/class-based-views/>) para que puedas ver la cantidad de vistas pre-definidas que nos ofrece Django para facilitarnos la escritura de código. Pero repetimos, hay que saber que hace cada vista, para saber usarla.

Ahora usaremos esta vista basada en clases para "posts" por lo que también debemos definirlo en la url, por lo que vamos a dirigirnos a "urls.py" de nuestra aplicación y cambiamos el "path" de la vista basada en funciones por la forma de llamar a una vista basada en clases y que pueda renderizar el html.

```
apps > posts > urls.py > ...
1  from django.urls import path
2  from .views import PostListView
3  from . import views
4
5
6  urlpatterns = [
7      path('posts/', PostListView.as_view(), name='posts'),
8  ]
9
```

Como puedes observar, importamos desde "views" la vista de clase "PostListView". En el "path" declaramos la ruta de acceso, luego la vista de clase (donde antes teníamos la vista en base a funciones) con la función ".as_view()" y, por último, el nombre para hacer referencia a la url.

No olvides borrar la vista basada en funciones que teníamos anteriormente en "views.py", ya que esta no la usaremos más.

Guardamos todo y si lo probamos en el navegador se tiene que ver de la misma forma que se veía antes.

Ahora es momento de crear la vista para un registro individual, es decir, para acceder a un post en particular, por lo que vamos a crear una view para el mismo, pero también debemos crear un nuevo html, el cual va a mostrar dicho post. Por lo que para comenzar a ordenar las plantillas que vamos a ir creando para lo que necesitemos mostrar. Entonces, vamos a crear una nueva carpeta dentro de la carpeta templates llamada "posts" para hacer referencia a estos.

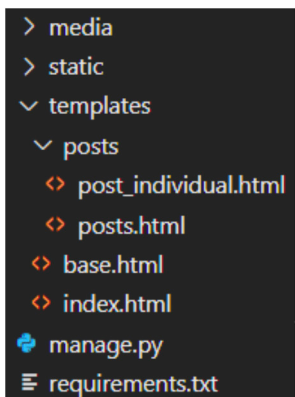
> Accediendo a un registro

Accediendo mediante el id a un registro en particular

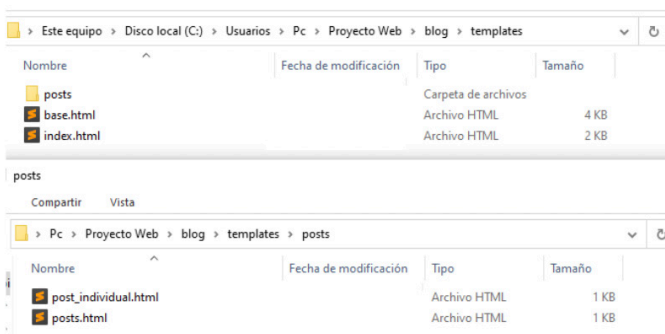


Pasos

Dentro de esta carpeta vamos a alojar el html para la vista del post individual y también vamos a mover el archivo "posts.html" que se encuentra en "templates" hacia la subcarpeta "posts" creada recientemente. La estructura quedaría así:



Como puedes observar, ahora los html "base" e "index" quedaron en la carpeta "templates" y "posts.html" y "post_individual.html" (que lo creamos) quedaron en la subcarpeta "posts". Así sería la vista desde el explorador de Windows:



Vamos a dirigirnos al views de nuestra aplicación para crear la vista basada en clases para nuestro post individual. Vas a ver también que debemos cambiar el acceso del template en la vista "PostListView" ya que cambiamos de carpeta a la plantilla "posts.html".

```
1 from .models import Post
2 from django.views.generic import ListView, DetailView
3
4 class PostListView(ListView):
5     model = Post
6     template_name = "posts/posts.html"
7     context_object_name = "posts"
8
9 class PostDetailView(DetailView):
10    model = Post
11    template_name = "posts/post_individual.html"
12    context_object_name = "posts"
13    pk_url_kwarg = "id"
14    queryset = Post.objects.all()
15
```

Para crear esta vista importamos la vista que nos provee Django "DetailView". Creamos la vista de clase "PostDetailView" que hereda de "DetailView", donde al atributo "model" le decimos que busque en el modelo "Post", al atributo "template_name" que renderice la plantilla "posts.html", el atributo "pk_url_kwarg" obtendrá el id del "post" y el atributo "queryset" se encarga de obtener todos los datos referidos al registro al que accedemos mediante el id obtenido. Lo próximo es definir la url de acceso.

```
apps > posts > urls.py > ...
1  from django.urls import path
2  from .views import PostListView, PostDetailView
3
4  urlpatterns = [
5      path('posts/', PostListView.as_view(), name='posts'),
6      path("posts/<int:id>/", PostDetailView.as_view(), name="post_individual"),
7  ]
```

Para esta vista pasamos un argumento más que es el id obtenido y lo declaramos como un entero.

Ahora cargaremos nuestro template "post_individual.html" para poder mostrar un post individual

```
templates > posts > post_individual.html > ...
1  <br>
2  <li>{{ posts.titulo }}</li>
3  <li>{{ posts.subtitulo }}</li>
4  <li>{{ posts.categoria }}</li>
5  <br>
6  <li>{{ posts.texto }}</li>
7
```

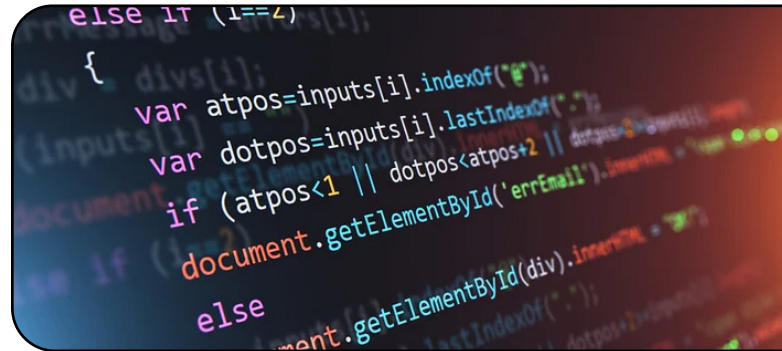
y la vista que tendríamos desde el navegador sería algo así (pusimos el id 2 para el ejemplo)



- Lorem Ipsum 2
- Neque porro quisquam est qui dolorem ipsum quia dolor sit amet, consectetur, adipisci velit...
- Categoría 1
- Cras lacus augue, rutrum ac enim non, tristique elementum arcu. Proin a sodales ex. Mauris pharetra posuere neque, vitae aliquet elit rhoncus eget. Nullam porttitor rhoncus lectus, eget tristique dui porttitor vel. Maecenas vel bibendum urna, vitae tristique urna. Aenean ultricies turpis ligula, eget cursus metus elementum et. Etiam congue magna ac imperdiet vestibulum. Suspendisse consequat sem non imperdiet aliquet. Suspendisse rutrum elementum tellus, vitae mollis odio ornare eu. Curabitur ut nisi vitae mauris tempus tempus ac quis nulla. Proin nisl metus, ultrices eu arcu eu, iaculis rutrum massa. Ut dignissim leo non sodales molestie. Vestibulum malesuada odio sed scelerisque porttitor. Donec rutrum tincidunt bibendum. Sed dapibus faucibus purus, sit amet lobortis odio volutpat ac. Fusce commodo feugiat enim, sed finibus nisi venenatis vel.

> Accediendo a un registro

Mostrando imagenes en el html



Pasos

Para acceder a mostrar la imagen que tiene cargada nuestro post, tendremos que usar la siguiente declaración en template:

```
{{ posts.imagen.url }}
```

Sin embargo, tenemos que realizar una modificación antes de poder usar esta llamada, ya que si la agregamos ahora, nos saldrá un ícono de imagen "rota" o "no encontrada". La modificación que tenemos que realizar es la de declarar en nuestra "urls.py" principal, el siguiente código:

```
blog > urls.py > ...
17 from django.contrib import admin
18 from django.urls import path, include
19 from .views import index
20 from django.conf import settings
21 from django.conf.urls.static import static
22 from django.contrib.staticfiles.urls import staticfiles_urlpatterns
23
24
25 urlpatterns = [
26     path('admin/', admin.site.urls),
27     path('', index, name='index'),
28     path('', include('apps.posts.urls')),
29 ]+static(settings.STATIC_URL, document_root=settings.STATIC_ROOT)
30 urlpatterns += staticfiles_urlpatterns()
31 urlpatterns += static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
32
```

Lo primero es importar
from django.conf.urls.static import static
y luego
from django.contrib.staticfiles.urls import staticfiles_urlpatterns

las cuales van a servir para hacer las declaraciones:

```
+static(settings.STATIC_URL, document_
root=settings.STATIC_ROOT)
urlpatterns += staticfiles_urlpatterns()
urlpatterns += static(settings.MEDIA_URL,
document_root=settings.MEDIA_ROOT)
```

que debes ponerla tal cual está, con el + en la misma línea de código en la que termina la declaración de las urls (línea 29 en este caso).

Ahora te explicamos para que sirve esto:

- La primera (29 en este caso) línea usa la función "static" para añadir una URL que sirve los archivos estáticos desde la ruta definida en "settings.STATIC_URL" y que se encuentran en el directorio definido en "settings.STATIC_ROOT1". Esta función solo se debe usar en desarrollo, ya que en producción se espera que los archivos estáticos sean servidos por un servidor web.
- La segunda línea (30 en este caso) usa la función "staticfiles_urlpatterns" para añadir las URLs que sirven los archivos estáticos desde las aplicaciones y los directorios definidos en "settings.STATICFILES_DIRS". Esta función es útil cuando se usa el sistema de archivos estáticos de Django, que permite recoger los archivos estáticos de cada aplicación y de otros lugares y colocarlos en un solo lugar que se pueda servir fácilmente.

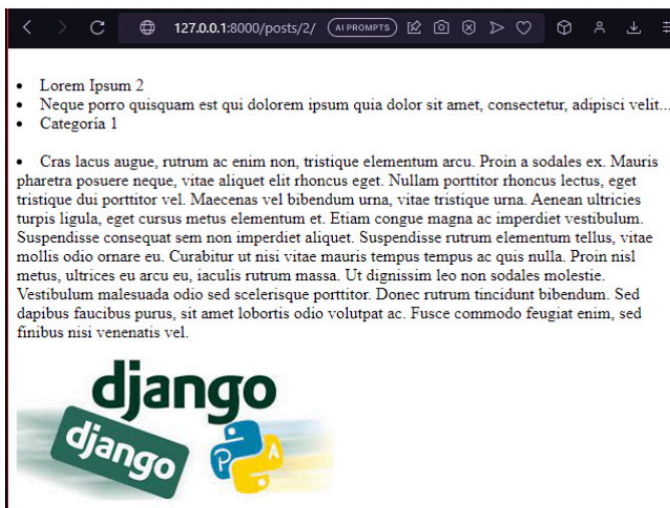
- La tercera línea (31 en este caso) usa de nuevo la función `static` para añadir una URL que sirve los archivos multimedia desde la ruta definida en "settings.MEDIA_URL" y que se encuentran en el directorio definido en "settings.MEDIA_ROOT". Estos archivos son los que se suben por los usuarios, como imágenes, vídeos, documentos, etc. Al igual que con los archivos estáticos, esta función solo se debe usar en desarrollo.

Ahora que todo está listo, ya que podemos acceder a los posts que están en nuestra base de datos y además podemos acceder a un post específico, sería momento de enlazar todo haciendo las referencias desde el "base.html" para que al hacer click en posts, podamos ver el listado de posts y si luego hacemos click en un post en particular, podamos ver el detalle de este.

Ahora podemos cargar en "post_individual.html" la etiqueta `{{ posts.imagen.url }}`

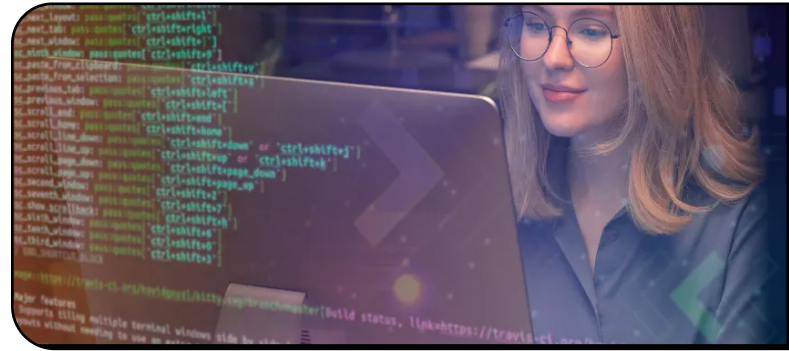
```
templates > posts > post_individual.html > img
1  <br>
2  <li>{{ posts.titulo }}</li>
3  <li>{{ posts.subtitulo }}</li>
4  <li>{{ posts.categoria }}</li>
5  <br>
6  <li>{{ posts.texto }}</li>
7  
```

Y en el navegador ya podremos ver la imagen del registro post al que estamos accediendo:



> Enlazando referencias

Agregando las referencias de acceso a nuestras urls



Pasos

Para esto, lo primero que vamos a hacer es declarar el nombre de nuestra aplicación en "urls.py" de nuestra aplicación:

```
apps > posts > urls.py > ...
1 from django.urls import path
2 from .views import PostListView, PostDetailView
3
4 app_name = 'apps.posts'
5
6 urlpatterns = [
7     path('posts/', PostListView.as_view(), name='posts'),
8     path("posts/<int:id>/", PostDetailView.as_view(), name="post_individual"),
9 ]
10
```

Hacemos esta declaración para que desde "base.html" (o cualquier otro html) podamos hacer referencia mediante la sintaxis de Django a la aplicación en particular a la que queremos acceder desde un link o botón para acceder y además le decimos a la url específica que vamos a acceder de la aplicación. Como puedes ver en este caso, tenemos dos urls declaradas y lo que queremos hacer es acceder a la url "posts" desde el Navbar, por lo que vamos a declarar en nuestro Navbar:

```
templates > base.html > html > body > header
33
34 <ul class="nav-links">
35 <li><a href="{% url 'index' %}">Inicio</a></li>
36 <li><a href="#">Acerca de nosotros</a></li>
37 <li><a href="{% url 'apps.posts:posts' %}">Posts</a></li>
38 <li><a href="#">Contacto</a></li>
39 <li><a href="#">Registro</a></li>
40 </ul>
41 </div>
42 </nav>
43
```

En esta estructura base de un Navbar declaramos en cada "botón" las referencias de acceso para cada plantilla en cuestión.

Hasta este momento del apunte tenemos creado el acceso de "index" desde el "urls.py" principal y ahora tenemos los accesos de la aplicación posts. A medida que vayamos creando más aplicaciones iremos haciendo más accesos y se irá completando nuestro Navbar y todos los accesos.

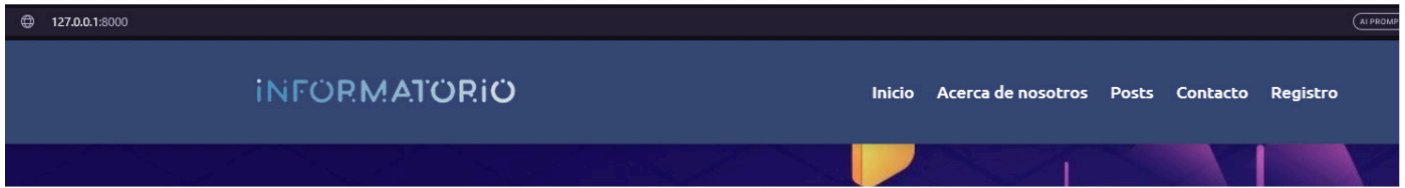
La forma en como se declara es básicamente con la nomenclatura que venimos usando. Para "index" será entonces
`{% url 'index' %}`

y para nuestra app posts será:

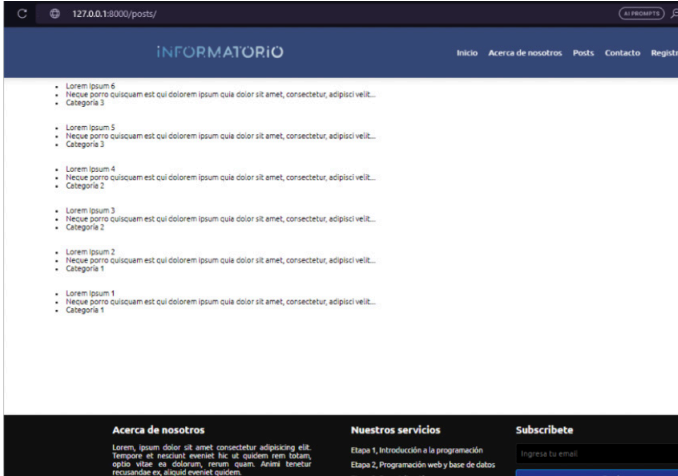
`{% url 'apps.posts:posts' %}`

donde apps se refiere a la carpeta donde está alojada nuestra aplicación, posts se refiere a la aplicación en sí, y :posts se refiere a la url creada para acceder a la vista que queremos, en este caso nos da el acceso a los posts alojados en la base de datos.

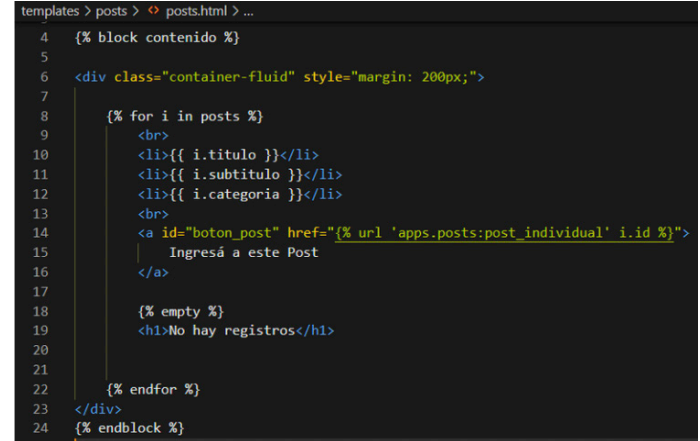
Ya podemos probarlo:



ingresamos a la página principal, damos click en Posts:



A la izquierda la captura del template "post.html" y a la derecha agregamos el botón:

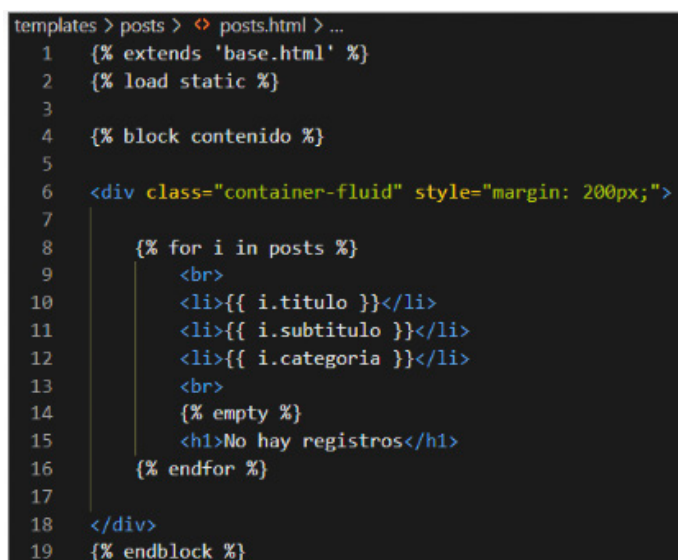


Ya accede y se ve correctamente los posts registrados.

Para que se aprecie mejor, hicimos herencia del template "base.html" tal cual ya lo habíamos hecho anteriormente.

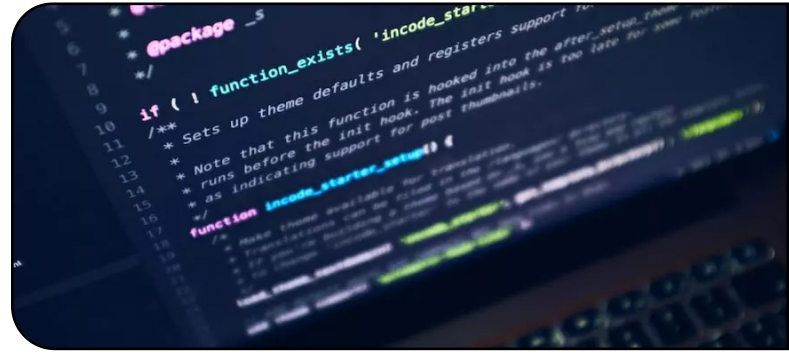
Te vamos a dejar igualmente la captura de esto.

Pero lo que nos falta ahora para poder acceder a las noticias es darle una referencia y esto lo podemos hacer con un botón o simplemente asignar la referencia al título, o subtítulo. Vamos a hacerlo con un botón.



> Muestra

Mostrando el resultado



Puedes ver que para ingresar al post específico lo referenciamos con `{% url 'apps.posts:post_individual' i.id %}` de manera tal que, a diferencia con la referencia al listado de posts, ahora ingresamos a través del id del post.

Agregamos algo de css (“boton_post”) para que puedas ver mejor la forma de un botón simple.

Esta sería la vista final:

Si seguiste los pasos hasta aquí (siempre respetando las sintaxis empleadas, los pasos correctos y guardando siempre cada cambio realizado) no deberías tener problemas y ver prácticamente como te estamos mostrando. Nosotros haremos los formatos con css y js para que se vaya viendo mejor. ¡Te invitamos a que también lo hagas! Luego te iremos mostrando que como verá nuestro blog.

Ahora comenzaremos con los formularios.

